PSAI_132

Ocelloids XCM Transfer Monitoring Service

- **Team Name:** SO/DA zone
- **Payment Address:** Fiat 04.05.2023, 16:37
- **Level:** 2

Project Overview

This grant proposal is a follow-up to the Ocelloids Monitoring SDK, previously delivered and available here: https://github.com/w3f/Grant-Milestone-Delivery/pull/934

The objective of this grant is to develop an open-source monitoring service using the Ocelloids SDK. This service will monitor XCM transfers across selected parachains. The primary purpose is to offer service providers integrating with a single chain (Asset Hub as starting point) and monitoring effects on other chains that are connected via HRMP and that use XCM as their message format. The service will support connecting to the configured networks through light clients in order to reduce infrastructure overhead for service providers. Users will have access to a self-hosted HTTP API to subscribe to XCM transfers and manage their subscriptions. A public Docker image will be published to facilitate service deployment.

Project Details

The service will support bidirectional XCM transfers, namely asset teleports and reserve-based transfers, between selected parachains.

The flow of the monitoring will work as follows:

1. The service will monitor on the origin chain for the event `xcmpqueue.xcmpMessageSent` associated to the extrinsic sent by accounts of interest. The service will extract the XCM message hash from this event.
2. The service will query the storage in `parachainSystem.hrmpOutboundMessages` at the block of the event to get all outbound messages and filter for recipient chain IDs that are supported. Subsequently it will decode the message data to get the set of XCM instructions to filter for combinations of instructions related to asset teleports or reserve-based transfers (i.e. `ReserveAssetDeposited`, `ReceiveTeleportedAsset`, `WithdrawAsset`, and `DepositAsset`). Then, it will get the `blake2-256` hash of the message data to match it with the message hash obtained in Step 1. The service will store a persistent task to be matched in subsequent steps.
3. At the destination chain, the service will monitor for the events `xcmpqueue.success` and `xcmpqueue.fail`. It will match the message hash extracted from these events with the message hash of the origin.
4. It will send a notification to the configured webhook to inform of the status of the XCM transfer along with contextual information. See section Notifications for details.

Before using the service, users will need to configure the supported networks and connection providers. An example configuration can be found in the section Service Configuration. We will support connections to the network using both RPC clients and light clients. Further details on supported Substrate clients can be found in the Supported Substrate Clients section.

The service will expose an HTTP API for users to add subscriptions, specifying the channels they want to monitor and the accounts involved. Users can create multiple subscriptions and modify or delete them as needed. Users can also set up webhook endpoints to receive notifications related to their transfers. Detailed information about the API will be provided in the Subscriptions API section.

To keep the project manageable, the current scope includes support for Asset Hub, Astar and Acala. In the future, we plan to expand support to include more chains.

Notifications

As previously mentioned, the monitoring service will accept webhook endpoints for delivering notifications. Initially, notifications will be provided for XCM message reception.

The following types of notifications correspond to different scenarios:

XCM Execution Success

The XCM message sent from the origin chain was received and executed successfully. In this case, we will send an “XCM Transfer Success” notification, including contextual data such as the XCM message hash, block numbers of the XCM message on the origin and destination chains, sender account, beneficiary account, transferred asset and amount.

XCM Execution Fail

The XCM message was received at the destination chain but failed to execute correctly. In this case, an “XCM Transfer Fail” notification will be sent, including the XCM error returned in the event and additional contextual data.

Service Configuration

The service will load a configuration file at startup, similar to the example provided below:

```
{
  "networks": [
    {
      "name": "assethub",
      "id": 1000,
      "relay": "polkadot",
      "provider": {
        "type": "ws",
        "url": "wss://asethub-rpc"
      }
    },
    {
      "name": "parallel",
      "id": 2012,
      "relay": "polkadot",
      "provider": {
        "type": "smoldot"
      }
    }
  ]
}
```

Other common service options, such as the listening port for the Subscriptions API, will be configurable using environment variables.

Subscriptions API

API Methods

`POST /api/subscriptions/{id}`

Creates a new subscription document. To avoid the need for the client to store a server generated identifier, we will require the client to provide a unique subscription identifier. This will also allow the client to use meaningful identifiers e.g. `1000-2004-cohort1`.

Example request body:

```
{
  "subject": "xcm-hrmp-transfers",
  "args":{
    "origin": {
      "network": "assethub",
      "senders": [
        {
          "accountId": "5GEWco3EGfM27Z9cAmVnzFnZ6Y7vkqNVvWM4NgQZj4n84Wh6",
          "type": "AccountId32"
        }
      ]
    },
    "destination": {
      "network": "moonbeam"
    },
  },
  "notification": {
```

```
        "type": "webhook",
        "url": "https://my-callback",
        "maxRetries": 50
      }
    }
```

`PATCH /api/subscriptions/{id}`

Accepts a JSON Patch operations array according to RFC6902

Example:

```
    [
      {
        "op": "add",
        "path": "/args/origin/senders/1",
        "value": {
          "accountId": "5HWSEZr3DQXaN4Tk2Y9pYyAPKWeu28P94qeWWgUZ4k2mrbGB",
          "type": "AccountId32"
        }
      }
    ]
```

The resulting patched document will be validated against the subscription document schema.

`DELETE /api/subscriptions/{id}`

Removes a subscription by ID.

`GET /api/subscriptions`

Returns the list of subscriptions.

`GET /api/subscriptions/{id}`

Returns the subscription document under the specified ID.

`PUT /api/subscription/{id}`

Replaces an existing subscription document by ID.

Supported Substrate Clients

We will support WebSocket RPC endpoints and light clients. However, please note that light client support may be limited due to its experimental nature.

During preliminary testing, we identified some limitations:

1. Using the Smoldot through Subtrate-Connect only support bootnodes configured with secure WebSocket connection. In our exploration, we've seen that only Astar and Acala have bootnodes with secure WebSocket connections. However, it is easy to overcome this limitation since Smoldot supports all the connection types. Reference: Github issue
2. Some runtimes, such as Moonbeam, currently cannot be compiled by the light client. Reference: Github issue

We will prioritize chains that are light client ready.

Storage

We will provide persistent storage for operational data, including subscription configurations, processed block number and hashes, XCM messages and notifications. We will use Level as the database abstraction backed by LevelDB.

Catch-up Mechanism

We will implement a mechanism to process missed blocks in case the monitoring service experiences downtime. For each processed block, its hash and block height will be saved in the database. When the service restarts, it will begin following the current finalized blocks. If the parent hash of the finalized block does not match the last-known hash stored in the database, the service will start processing the missing parent hash blocks until all missing ones are processed.

Management Tools

Since notifications and XCM message matching tasks are stored in the database and retried until success, we will provide a method to clear pending states. This is crucial to prevent indefinite retries of pending tasks. For example, if a webhook endpoint is changed while a notification is pending, it may never succeed.

We will supply a script enabling administrators to inspect and remove XCM message matching and notification tasks from the database.

Tech Stack

- Typescript
- Node.js
- Polkadot.js API
- Smoldot
- Fastify
- Level

Ecosystem Fit

This project aligns well with the growing demand for robust tooling and infrastructure necessary to fulfill Asset Hub's vision of becoming the main place for asset management within the Polkadot ecosystem. To expand its user base, Asset Hub must provide a solution that abstracts away the technical intricacies of cross-chain transfers and enhances the user experience for managing assets across the expansive Polkadot ecosystem.

The Integrations Team at Parity is already taking initial steps toward achieving this goal by working on the Assets Transfer API and the NoS launch script. These efforts target custodians, exchanges, and institutions, simplifying integration with Asset Hub and the construction of XCM messages. The importance of these tools, along with the overarching vision for Asset Hub, is comprehensively outlined in Iker's Polkadot forum post.

The addition of the Ocelloids XCM monitoring service will enhance this toolkit, providing institutions with a straightforward method to track their cross-chain transfers. This corresponds to step 4 of the "Withdrawals User Journey Example" in Iker's post, as detailed here.

With the inclusion of support for light clients in the service, institutions will experience a substantial reduction in infrastructure overhead since they will no longer need to run parachain nodes to monitor their transfers.

Furthermore, the demand for a generalized monitoring solution with XCM support has been highlighted in discussions within the Polkadot community, as illustrated in this Polkadot forum post: Generalized multichain monitoring solution.

Team :busts_in_silhouette:

Team members

- Marc Fornós
- Xueying Wang

Contact

- **Contact Name:** Marc Fornós
- **Contact Email:** frozen.pomelo@gmail.com

Legal Structure

N/A

Team's experience

Marc Fornós has been designing and implementing software systems for 20 years. He is an expert in the area of distributed systems and data-intensive applications. His experience ranges from warehouse automation with radio-frequency terminals, to being the technical director of an airline post-sale revenue optimization software-as-a-service platform, generating millions in incremental revenue during eight years of operation. In the recent past, he was in charge of evolving a commercial Ethereum block

explorer and bootstrapping an explorer for Substrate-based networks focused on the contracts pallet and ink! smart contracts.

Xueying Wang pivoted to software development after completing an MSc. in Aerospace Engineering and has been in the industry for the past eight years. During this time, she pioneered conversational AI assistants for airlines, counting more than 20 assistants in production covering ten languages for customer service, FAQ and in-chat purchases. She also built a scalable publish-subscribe system to trigger actions on flight feed events for the automated agents. She participated in designing a composable Solid POD/RDF data browser and bootstrapping an explorer for Substrate-based networks focused on the contracts pallet and ink! smart contracts.

Team Code Repos

- https://github.com/sodazone
- https://github.com/mfornos
- https://github.com/XY-Wang
- https://github.com/frozenpomelo (shared account)

Development Roadmap :nut_and_bolt:

Overview

- **Total Estimated Duration:** 2 months
- **Full-Time Equivalent (FTE):** 2
- **Total Costs:** 28,000 EUR

Milestone 1

- **Estimated duration:** 2 months
- **FTE:** 2
- **Costs:** 28,000 EUR

Number	Deliverable	Specification
0a.	License	Apache 2.0
0b.	Documentation	We will provide both inline documentation of the code and a basic guide that explains how to set up and run the monitoring service.
0c.	Testing and Testing Guide	Core functions will be fully covered by comprehensive unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests.
0d.	Docker	We will provide a Dockerfile to ease the deployment and execution of the service. A Docker image of the service will be published in Docker Hub and Github Container Repository.
1.	XCM Monitoring Service	The XCM monitoring service that supports asset teleports and reserve-based transfers between the following parachains: Asset Hub, Astar and Acala. The service will feature what was described in Project Details.
2.	Management Tools	Administrator scripts to inspect and delete pending XCM messages matching and notification tasks, as described in Management Tools.

Future Plans

For the XCM monitoring service, we have plans to expand its capabilities and reach which include:

1. **Support for More Networks:** We plan to broaden the range of networks supported by the XCM monitoring service, enabling a more extensive and inclusive monitoring ecosystem.

2. **Support for More XCM Protocols:** We will add support for XCMP when ready.

3. **Enhanced Notifications:** Depending on user requirements and community feedback, we will extend our notification capabilities. This may involve providing notifications for asset transfers' initiation, such as when the block containing the XCM transfer is finalized on the origin chain. We are also considering notifications for when XCM messages sent through HRMP are processed on intermediate chains.

Our long-term vision for Ocelloids extends beyond just monitoring XCM transfers. We aim to create a hassle-free, comprehensive monitoring portal for Substrate networks and smart contracts within the ecosystem. This portal will offer a set of advanced features, including:

- Ready-to-use monitoring logic, i.e. monitoring programs, for the entire ecosystem. The XCM monitoring logic built through this grant could be one such monitoring program available.
- Marketplace for monitoring programs where users can subscribe to the programs that correspond to their needs.
- Advanced capabilities: anomaly and attack detection, machine learning, and forecasting.
- Real-time actionable insights on network, contract performance, compliance, and security.
- Multiple data centres for global, high-availability monitoring.
- Seamless integration with existing systems like Prometheus and SIEM.

In summary, our future plans encompass not only expanding the technical capabilities of the XCM monitoring service but also positioning as a central hub for monitoring and managing Substrate networks and smart contracts within the broader multi-chain ecosystem.